

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 13. Dictionary

Lecture: 140. Looping over Dictionary

Section 1: Lecture Summary

The lecture covers several important dictionary methods in Python that facilitate looping over a dictionary's entries and safely accessing or modifying values. The instructor first recaps how to iterate over dictionary keys using a simple `for` loop and then introduces dictionary class methods such as `keys()`, `values()`, and `items()` that provide specialized iterables for keys, values, or key-value pairs. The lecture then explains the difference between using square bracket indexing versus the `get()` method for retrieving values, particularly in handling missing keys. Finally, the concept of `setdefault()` is discussed as a way to retrieve a value for a key and simultaneously insert a default value into the dictionary if the key does not exist. Example code snippets are demonstrated throughout using a dictionary of numeric keys and string values.

Section 2: Key Concepts and Explanations

- Iterating over a Dictionary

- Using a simple `for` loop over the dictionary itself iterates over the keys.
- Accessing values during iteration: For key `k`, value is retrieved by `dict[k]`.

- Dictionary Methods for Looping

- `keys()`: Returns a `dict_keys` object containing all keys.
 - Can be iterated over using a `for` loop.
 - Example: `for k in dict.keys():`
- `values()`: Returns a `dict_values` object containing all the dictionary's values.
 - Requires iteration to access individual values.
 - Example: `for v in dict.values():`
- `items()`: Returns a `dict_items` object containing all key-value pairs as tuples.
 - Useful for looping over both keys and values simultaneously.
 - Example: `for k, v in dict.items():`

- Accessing Values: Indexing vs. `.get()` Method

- Square bracket indexing (`dict[key]`) retrieves the value if the key exists but raises a `KeyError` if it doesn't.
- `.get(key, default=None)` returns the value if key exists; otherwise returns `None` or the specified `default` value.
- `.get()` is useful for safe lookup without the need to handle exceptions.

- `.setdefault(key, default=None)` Method

- Returns the value for `key` if present.
- If `key` is not in the dictionary, inserts `key` with the provided `default` value and returns that value.
- If no default is provided, inserts `key` with `None`.
- Helps to insert a default entry while retrieving the value, often useful in grouping or counting tasks.

Section 3: Example Code and Use Cases

Create a sample dictionary

Iterating over dictionary keys (simple for loop)

Printing keys and values using indexing

Output:

1 One

2 Two

3 Three

4 Four

Using keys() method to loop over keys

Output: 1 2 3 4 (each printed on separate lines)

Using keys() with value access

Output:

1 One

2 Two

3 Three

4 Four

Using values() method to loop over all values

Output:

One

Two

Three

Four

Using items() to loop over key-value pairs

Output:

1 One

2 Two

3 Three

4 Four

Accessing value with get()

Accessing value with index operator (throws error if key not present)

`print(d1[5])` Raises `KeyError`

Using `setdefault()` with existing key

Using `setdefault()` to add a new key with default `None`

Output: {1: 'One', 2: 'Two', 3: 'Three', 4: 'Four', 5: None}

Using `setdefault()` to add new key with specified default value

Output: {1: 'One', 2: 'Two', 3: 'Three', 4: 'Four', 5: None, 6: 'Undefined'}

```
d1 = {1: 'One', 2: 'Two', 3: 'Three', 4: 'Four'}
```

```
for i in d1:
    print(i)    # prints keys: 1 2 3 4
```

```
for i in d1:
    print(i, d1[i])
```

```
for x in d1.keys():
    print(x)
```

```
for x in d1.keys():
    print(x, d1[x])
```

```
for v in d1.values():
    print(v)
```

```
for key, value in d1.items():
    print(key, value)
```

```
print(d1.get(4))            # Output: 'Four'
```

```
print(d1.get(5))            # Output: None (no error)
```

```
print(d1.get(5, 'Missing')) # Output: 'Missing'
```

```
print(d1.setdefault(3))    # Output: 'Three'
```

```
d1.setdefault(5)
print(d1)

d1.setdefault(6, 'Undefined')
print(d1)
```

Section 4: Key Takeaways

- Iterating directly over a dictionary iterates over keys only.
- ``keys()``, ``values()``, and ``items()`` methods provide explicit iterable views over dictionary keys, values, and key-value pairs respectively.
- Use ``get()`` method to safely retrieve a value for a key without risking a ``KeyError``; can supply an optional default value.
- The ``setdefault()`` method retrieves the value for a key and inserts that key with a default value if it does not exist in the dictionary.
- Understanding these methods enhances dictionary traversal and value access, making Python dictionary operations safer and more expressive.